

Nội dung ôn tập

Môn cấu trúc dữ liệu và giải thuật

I. Nội dung

1. **Phân tích thuật toán & độ phức tạp**
 - Đếm số phép toán cơ bản của một đoạn code (thường là 1–2 vòng lặp lồng nhau, có if).
 - Suy ra độ phức tạp tiệm cận: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, ...
2. **Cấu trúc dữ liệu lý thuyết + hàm thao tác**
 - Khái niệm + ví dụ + hình vẽ:
 - Stack (ngăn xếp), Queue (hàng đợi).
 - Danh sách liên kết đơn / kép.
 - Cây nhị phân, cây tìm kiếm nhị phân (BST), cây tổng quát.
 - Vector, bảng băm, heap.
 - Viết/mô tả thuật toán cho:
 - Chuyển trung tố $\rightarrow$ hậu tố, tính giá trị hậu tố.
 - Đổi cơ số, kiểm tra ngoặc đúng/sai.
 - Các hàm: find, insert, erase, count, sum, high (chiều cao), count_leaf...
3. **Lập trình hướng đối tượng + STL**
 - Viết **class** mô tả một đối tượng thực tế:
 - Sinh viên, thí sinh, nhân viên, xe máy, laptop,...
 - Các thuộc tính cơ bản + toán tử $>>$, $<<$.
 - Viết **class quản lý danh sách** bằng:
 - `std::vector` hoặc `std::list`:
 - Thêm phần tử, xóa theo khóa (mã, BKS,...).
 - In danh sách.
 - Tìm kiếm tuyến tính / nhị phân.
 - Thống kê (tổng giá trị, điểm cao nhất,...).
4. **Thuật toán sắp xếp**
 - Trình bày **ý tưởng** + **code**:
 - BubbleSort, SelectionSort, InsertionSort.
 - MergeSort, QuickSort, HeapSort.
 - Thường chỉ yêu cầu 1 thuật toán trong số đó.
5. **Mô phỏng / truy vết**
 - Mô tả quá trình:
 - Một thuật toán sắp xếp hoạt động trên dãy cho trước (Bubble/Selection/Insertion).
 - Bước Partition trong QuickSort.
 - Bước Merge trong MergeSort.
 - Vun đống (heapify) trong HeapSort.
 - Xây cây BST từ dãy khóa cho trước.
 - Vẽ bảng băm khi chèn các khóa kèm hàm băm.

Gợi ý:

- Nắm chắc **đếm vòng lặp $\rightarrow$ độ phức tạp**.
- Ôn lại **định nghĩa + cách biểu diễn + các phép toán cơ bản** cho từng cấu trúc DL.
- Luyện **viết class + dùng STL** theo “mẫu chung” rồi thay đổi tượng (SV, NV, Laptop, Xe máy...).
- Học **thuật toán sắp xếp & tìm kiếm** bằng cách:
 - Viết code,
 - Tự tay mô phỏng vài dãy số.

II. MỘT SỐ ĐỀ LUYỆN TẬP

ĐỀ LUYỆN TẬP SỐ 1

Câu 1 (1.5đ) – Phân tích độ phức tạp

Xét thuật toán:

```
int k = 0;
for (int i = 1; i <= n; i++) {
    int j = 1;
    while (j <= n) {
        if ((i + j) % 2 == 0) k++;
        j *= 2;
    }
}
```

1. Tính số phép so sánh trong **vòng while** được thực hiện nhiều nhất.
2. Suy ra độ phức tạp tiệm cận theo n .

Câu 2 (2.5đ) – Stack & chuyển trung tố $\rightarrow$ hậu tố

1. Trình bày:
 - o Khái niệm **ngăn xếp (stack)**.
 - o Các thao tác cơ bản: push, pop, top, empty.
2. Mô tả thuật toán (không cần code C++) để chuyển một biểu thức trung tố sang hậu tố.
3. Áp dụng, chuyển biểu thức sau sang dạng hậu tố:
$$((A + B) * C - D) / (E + F)$$

Câu 3 (3.0đ) – Lớp SảnPhẩm + quản lý bằng vector

1. Xây dựng lớp SanPham với các thuộc tính:
 - o string ma;
 - o string ten;
 - o double gia;
 - o int soLuong;
 - o Cài đặt toán tử nhập >>, xuất <<.
2. Xây dựng lớp QuanLySanPham dùng vector<SanPham> với các phương thức:
 - o void them(const SanPham &sp);
 - o bool xoaTheoMa(const string &ma);
 - o void inDanhSach() const;
 - o double tongGiaTriTonKho() const; (tính $\sum gia * soLuong$).
 - o SanPham* timTuyenTinh(const string &ma); (tìm kiếm tuần tự theo mã).

Câu 4 (2.0đ) – MergeSort

1. Trình bày ý tưởng MergeSort.
2. Viết hàm void mergeSort(int a[], int l, int r); sắp xếp tăng dần.

Câu 5 (1.0đ) – Truy vết Selection Sort

Cho dãy:

29, 10, 14, 37, 13

Mô tả trạng thái dãy sau mỗi lần chọn và đổi chỗ ở Selection Sort (sắp tăng dần).

ĐỀ LUYỆN TẬP SỐ 2

Câu 1 – Độ phức tạp

```
int s = 0;
for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j *= 3)
        s += i + j;
```

1. Ước lượng số lần câu lệnh $s += i + j$; được thực hiện.
2. Độ phức tạp tiệm cận?

Câu 2 – Queue & bài toán mô phỏng

1. Trình bày:
 - Khái niệm **hàng đợi (queue)**, các thao tác push, pop, front, empty.
2. Đề xuất cách dùng `queue<int>` (STL C++) để mô phỏng **quầy bán vé xem phim**:
 - Có n khách đến theo thứ tự.
 - Mỗi khách có mã số (1..n).
 - Hệ thống phục vụ lần lượt theo thứ tự đến.

Câu 3 – Lớp SinhVien + vector + tìm kiếm nhị phân

1. Viết lớp `SinhVien` với các thuộc tính: `ma`, `hoTen`, `namSinh`, `lop`, toán tử `>>`, `<<`.
2. Lớp `QLSV` dùng `vector<SinhVien>`:
 - `void them(const SinhVien &sv);`
 - `void xoaTheoMa(const string &ma);`
 - `void in() const;`
 - `SinhVien* timNhiPhan(const string &ma);` (giả sử vector đã **sắp xếp tăng dần theo mã**).

Câu 4 – QuickSort

1. Nêu ý tưởng QuickSort.
2. Viết hàm QuickSort sắp tăng dần mảng `int a[]` trên đoạn `[L, R]` (dùng partition đơn giản: `pivot = a[(L+R)/2]`).

Câu 5 – Bubble Sort

Cho dãy:

9, 4, 7, 1, 3

Truy vết Bubble Sort (sắp tăng) **sau mỗi vòng lặp**

ĐỀ LUYỆN TẬP SỐ 3

Câu 1 – Độ phức tạp

```
int c = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        c++;
```

1. Tính tổng số lần c++ được thực hiện.
2. Độ phức tạp?

Câu 2 – Danh sách liên kết đơn: push_front, erase(x)

Giả sử:

```
template <class T>
struct Node {
    T elem;
    Node *next;
};
template <class T>
class SList {
    Node<T> *Head, *Tail;
    int n;
public:
    SList(): Head(nullptr), Tail(nullptr), n(0) {}
    void push_front(T x);
    void erase(T x); // xoa nut dau tien co gia tri x (neu co)
};
```

Viết thân hàm push_front và erase.

Câu 3 – Cây tìm kiếm nhị phân (BST)

1. Nêu khái niệm cây tìm kiếm nhị phân.
2. Với Node:

```
template <class T>
struct Node {
    T elem;
    Node *left, *right;
};
```

Viết hàm:

- Node<T>* insert(Node<T>* root, T x); (trả về root mới).
- Node<T>* find(Node<T>* root, T x);

Câu 4 – HeapSort: ý tưởng + code khung

1. Trình bày ý tưởng HeapSort.
2. Viết các hàm:
 - void heapify(int a[], int n, int i);
 - void heapSort(int a[], int n); sắp tăng dần.

Câu 5 – Vun đống (heapify) một lần

Cho mảng (dạng cây nhị phân lưu trong mảng, index từ 0):

[10, 3, 8, 2, 5, 7]

Xem đây là một **max-heap bị sai tại gốc** (chỉ cần heapify tại i = 0, n=6).

Mô tả các bước của heapify(a, 6, 0).

ĐỀ LUYỆN TẬP SỐ 4

Câu 1 – Độ phức tạp với ma trận

```
double maxCol = 0;
for (int j = 0; j < n; j++) {
    double sum = 0;
    for (int i = 0; i < n; i += 2)
        sum += fabs(a[i][j]);
    if (sum > maxCol) maxCol = sum;
}
```

1. Số lần tính $\text{fabs}(a[i][j])$?
2. Độ phức tạp?

Câu 2 – Cây tổng quát: $\text{count}(x)$, height

```
template <class T>
struct Node {
    T elem;
    vector<Node<T>*> child;
};
```

1. Viết hàm đệ quy $\text{int count(Node<T>* H, T x)$; đếm số node có giá trị x .
2. Viết hàm $\text{int height(Node<T>* H)$; trả về chiều cao cây (cây một node có chiều cao 1; $\text{nullptr} \rightarrow 0$).

Câu 3 – Danh sách liên kết kép

```
template <class T>
struct DNode {
    T elem;
    DNode *prev, *next;
};
```

```
template <class T>
class DList {
    DNode<T> *Head, *Tail;
    int n;
public:
    void push_front(T x);
    void erase(T x); // xoa nut dau tien co gia tri x
};
```

Viết thân hàm push_front và erase .

Câu 4 – Bubble Sort (code)

Viết hàm:

```
void bubbleSort(int a[], int n);
```

Sắp xếp tăng dần.

Câu 5 – Tìm kiếm nhị phân (đệ quy)

Viết hàm:

```
bool binary_search(int a[], int L, int R, int x);
```

Mảng a đã sắp xếp tăng.

ĐỀ LUYỆN TẬP SỐ 5

Câu 1 – Độ phức tạp

```
int t = 0;
for (int i = 1; i <= n; i *= 2)
    for (int j = 1; j <= n; j++)
        if ((i * j) % 3 == 0) t++;
```

1. Ước lượng số lần lệnh trong if được xét.
2. Độ phức tạp?

Câu 2 – Lớp Vector tự cài đặt

Giả sử:

```
template <class T>
class MyVector {
    T *buffer;
    int n, cap;
public:
    void erase(int k); // xoa phan tu vi tri k (0 <= k < n)
    void insert(int k, const T &x); // chen x vao vi tri k
};
```

Viết thân 2 hàm (bỏ qua việc tăng cap cho đơn giản, giả sử còn chỗ).

Câu 3 – Stack kiểm tra ngoặc

1. Nêu ý tưởng dùng stack<char> để kiểm tra biểu thức ngoặc ()[]{} có hợp lệ không.
2. Viết hàm:

```
bool checkBracket(const string &s);
```

Câu 4 – Selection Sort (code)

Viết hàm Selection Sort sắp tăng dần:

```
void selectionSort(int a[], int n);
```

Câu 5 – Bảng băm (hash table)

Cho kích thước bảng băm $m = 10$, hàm băm:

$$h(k) = k \bmod 10$$

Chèn lần lượt các khóa:

23, 44, 12, 39, 25, 54

Giả sử xử lý va chạm bằng **tuyến tính (linear probing)**: nếu ô $h(k)$ đã đầy thì thử $h(k) + 1, h(k) + 2, \dots \pmod m$.

Hãy mô tả trạng thái bảng sau khi chèn xong.